



ARYANS

**TAKNEEK 2026**

---

# **The Big-GEM Theory**



**Project:** The Big-GEM Theory  
**Event:** Takneek 2026 – Zenith  
**Team:** Aryans  
**Base project:** NVIDIA GEM GPU RTL simulator  
**Main work:** Native GPU support for DSP48E2, CARRY4, and SRLC32E  
**Repository:** <https://github.com/Pratham-015/GEM>

## Contents

|          |                                                           |           |
|----------|-----------------------------------------------------------|-----------|
| <b>1</b> | <b>Project Summary</b>                                    | <b>1</b>  |
| 1.1      | Aim of the project . . . . .                              | 1         |
| 1.2      | Main result . . . . .                                     | 1         |
| <b>2</b> | <b>Problem Statement and Project Scope</b>                | <b>2</b>  |
| 2.1      | Required work . . . . .                                   | 2         |
| 2.2      | Limits taken from the problem statement . . . . .         | 2         |
| 2.3      | Evidence for the requested technical areas . . . . .      | 2         |
| <b>3</b> | <b>Overall Design</b>                                     | <b>4</b>  |
| 3.1      | Data flow . . . . .                                       | 4         |
| 3.2      | Main source files . . . . .                               | 4         |
| <b>4</b> | <b>Deliverable A: Host Parser and Yosys Pre-Processor</b> | <b>5</b>  |
| 4.1      | Keeping macros during synthesis . . . . .                 | 5         |
| 4.2      | Typed macro records . . . . .                             | 5         |
| 4.3      | Host memory layout . . . . .                              | 5         |
| 4.4      | Host formatter and GPU memory hierarchy . . . . .         | 6         |
| <b>5</b> | <b>Deliverable B: CUDA Engine and Boomerang Changes</b>   | <b>7</b>  |
| 5.1      | Macro dependency levels . . . . .                         | 7         |
| 5.2      | Work done in one simulation cycle . . . . .               | 7         |
| 5.3      | Reducing warp divergence . . . . .                        | 8         |
| <b>6</b> | <b>Deliverable C: Hardware Macro Models</b>               | <b>9</b>  |
| 6.1      | DSP48E2 subset . . . . .                                  | 9         |
| 6.2      | CARRY4 and fused carry chains . . . . .                   | 9         |
| 6.3      | SRLC32E . . . . .                                         | 9         |
| 6.4      | C++ and CUDA implementation files . . . . .               | 9         |
| <b>7</b> | <b>Verification Work</b>                                  | <b>11</b> |

---

|           |                                                           |           |
|-----------|-----------------------------------------------------------|-----------|
| 7.1       | Levels of checking . . . . .                              | 11        |
| 7.2       | Saved test results . . . . .                              | 11        |
| <b>8</b>  | <b>Deliverable D: Benchmarks and Performance Analysis</b> | <b>12</b> |
| 8.1       | Test method . . . . .                                     | 12        |
| 8.2       | Modified GEM throughput . . . . .                         | 12        |
| 8.3       | Fair upstream Boolean comparison . . . . .                | 12        |
| 8.4       | CARRY4 bottleneck work . . . . .                          | 13        |
| 8.5       | Nsight Compute results . . . . .                          | 13        |
| 8.6       | Numerical analysis . . . . .                              | 14        |
| <b>9</b>  | <b>Deliverable E: Documentation and Repository Work</b>   | <b>15</b> |
| 9.1       | Documentation added . . . . .                             | 15        |
| <b>10</b> | <b>Build and Reproduction Steps</b>                       | <b>15</b> |
| 10.1      | Required tools . . . . .                                  | 15        |
| 10.2      | Build . . . . .                                           | 15        |
| 10.3      | Run verification . . . . .                                | 15        |
| 10.4      | Run benchmarks . . . . .                                  | 15        |
| <b>11</b> | <b>Performance limits</b>                                 | <b>16</b> |
| <b>12</b> | <b>Conclusion</b>                                         | <b>16</b> |
| <b>13</b> | <b>References</b>                                         | <b>16</b> |
| <b>A</b>  | <b>Deliverable-to-File Map</b>                            | <b>17</b> |
| <b>B</b>  | <b>Evidence Folders</b>                                   | <b>17</b> |

## 1. Project Summary

---

### 1.1 Aim of the project

NVIDIA GEM normally converts RTL circuits into one-bit Boolean gates before running them on the GPU. This works well for normal logic, but large hardware blocks lose their original form during synthesis. A multiplier, carry chain, or shift-register LUT can turn into many small gates. This adds more graph nodes and more work for the simulator.

Our project adds a second path for three Xilinx primitives:

- DSP48E2, using the subset fixed by the problem statement;
- CARRY4, including connected carry chains; and
- SRLC32E, with its clocked 32-bit state and two read outputs.

These blocks are kept as named cells during Yosys synthesis. The host code reads their ports, puts them in the graph, assigns GPU memory, and sends a macro program to the CUDA kernel. The kernel then runs the original Boolean work and the new word-level work in one simulation flow.

### 1.2 Main result

The required flow is present from SystemVerilog input to CUDA output. The three macro models pass the saved CPU, Verilog, Xilinx UNISIM, and GPU comparisons. The saved mixed-graph tests also pass with no reported mismatches.

The performance work is not complete. The fair Boolean-only comparison gives  $0.893\times$  the upstream speed, which is a 10.67% slowdown. The CARRY4 experiment improved from  $0.117\times$  to  $0.222\times$ , but it is still slower than the shredded upstream circuit. These limits are stated clearly in this report.

## 2. Problem Statement and Project Scope

### 2.1 Required work

The Zenith problem asks for five main deliverables. The first three are the implementation, the fourth is performance testing, and the fifth is the report.

Table 1: Present status of the problem-statement deliverables.

| Part | Deliverable                         | Work present in the repository                                                                                                          |
|------|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| A    | Host parser and Yosys pre-processor | Macro-preserving synthesis, DSP control mapping, typed macro ports, graph links, and 64-bit memory layout.                              |
| B    | CUDA engine and Boomerang changes   | Macro program, level order, shared-memory staging, word-level evaluation, state update, and Boolean-only path.                          |
| C    | Hardware macro models               | DSP48E2 subset, CARRY4/fused carry chain, and SRLC32E models with saved differential tests.                                             |
| D    | Benchmarks and analysis             | CPS tests, upstream Boolean comparison, CARRY4 experiment, and Nsight data are present. Current results do not show an overall speedup. |
| E    | Documentation                       | Setup notes, benchmark instructions, saved evidence, and this report are present.                                                       |

### 2.2 Limits taken from the problem statement

The implementation follows these fixed limits:

- There is one global clock domain.
- DSP input and internal registers are disabled. Only PREG is enabled.
- Macro state starts at zero.
- DSP overflow and underflow outputs are not used.
- Yosys 0.68 with SystemVerilog support is required.
- The main macro arithmetic is implemented in the custom CUDA code.

### 2.3 Evidence for the requested technical areas

The following table connects the detailed points in the problem statement to the implementation. The code mechanisms for points 1–8 are present. Numerical analysis is also present, but the requested throughput gain has not yet been reached. Some of this work was not explained clearly in the earlier report.

Table 2: Location of the nine requested technical areas.

| No. | Required area                 | Evidence in this project                                                                                                                                                                                                                                                    |
|-----|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1   | Boomerang scheduler extension | <code>src/aig.rs</code> builds same-cycle and clock-boundary edges. <code>src/flatten.rs</code> groups macros by type and level. <code>csrc/kernel_v1_impl.cuh</code> runs Boolean stages and typed macro levels. The repeated grid barriers are still a performance limit. |

| No. | Required area                     | Evidence in this project                                                                                                                                                                                        |
|-----|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2   | Heterogeneous memory allocator    | <code>src/macro_layout.rs</code> creates separate, warp-padded 64-bit state and I/O areas while normal AIG state remains packed in 32-bit words.                                                                |
| 3   | Native macro evaluation           | <code>csrc/gem_macros.cuh</code> uses native signed and unsigned 64-bit values for carry, DSP, and SRL calculations. Masked selection is used for DSP state and SRL enable.                                     |
| 4   | Host-side memory formatter        | <code>src/flatten.rs</code> turns parsed macro nodes into flat descriptors and creates the two 64-bit GPU buffers passed by <code>src/bin/cuda_test.rs</code> .                                                 |
| 5   | Shared memory and warp operations | The production kernel stages six 64-bit fields per thread in shared memory. It uses <code>__ballot_sync</code> , <code>__all_sync</code> , and the existing Boomerang <code>__shfl_down_sync</code> operations. |
| 6   | Cycle-accurate C++/CUDA model     | The model is in <code>csrc/gem_macros.cuh</code> ; the CUDA runner is <code>verif/csrc/test_gem_macros.cu</code> . Sequential DSP and SRL state changes are checked at each rising edge.                        |
| 7   | Modified scheduling equations     | Section 5.1 gives the level and typed-cohort equations used by the host and GPU.                                                                                                                                |
| 8   | Primitive-to-memory diagram       | Figure 2 and Table 4 show the global, shared, and register placement.                                                                                                                                           |
| 9   | Numerical throughput analysis     | Deliverable D gives seven-sample medians, variation, paired upstream results, profiler measurements, and the remaining bottlenecks. The measurements currently show slowdowns, not an overall gain.             |

### 3. Overall Design

#### 3.1 Data flow

Figure 1 shows the path used by the project. It is the same path used by the saved end-to-end tests.

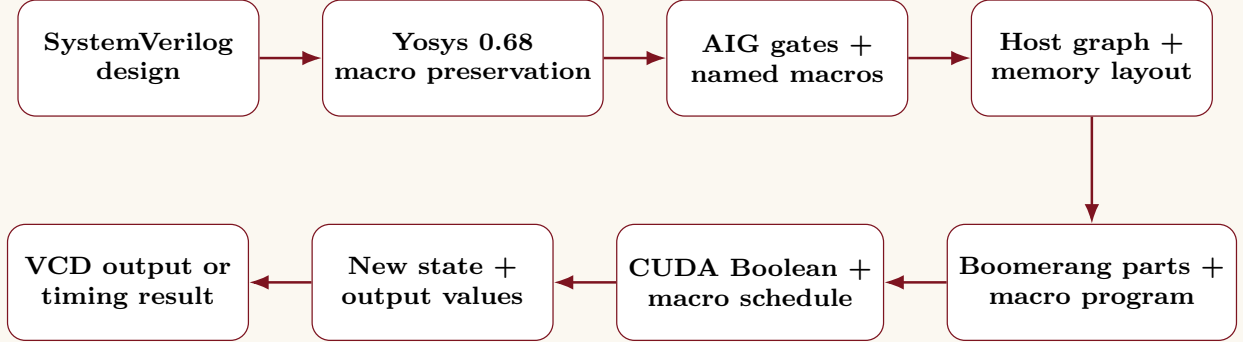


Figure 1: Big-GEM processing flow.

#### 3.2 Main source files

Table 3: Main files used by the implementation.

| File                                      | Purpose                                                                            |
|-------------------------------------------|------------------------------------------------------------------------------------|
| <code>scripts/synthesize_macros.py</code> | Runs Yosys, keeps the required macros, applies the DSP map, and checks the result. |
| <code>aigpdk/aigpdk_macros_bb.v</code>    | Declares the three macros as cells that Yosys must keep.                           |
| <code>aigpdk/dsp48e2_ps_map.v</code>      | Changes a supported DSP48E2 into the smaller control form used by GEM.             |
| <code>src/aig.rs</code>                   | Reads macro cells and adds their inputs, outputs, clocks, and graph links.         |
| <code>src/macro_layout.rs</code>          | Defines macro kinds, ports, dependency timing, and 64-bit GPU memory offsets.      |
| <code>src/flatten.rs</code>               | Builds the flat macro program used by the CUDA kernel.                             |
| <code>csrc/gem_macros.cuh</code>          | Contains the word-level CARRY4, DSP48E2, and SRLC32E functions.                    |
| <code>csrc/kernel_v1_impl.cuh</code>      | Adds macro gather, execution, output write, and state update steps to GEM.         |

## 4. Deliverable A: Host Parser and Yosys Pre-Processor

### 4.1 Keeping macros during synthesis

The synthesis script loads black-box declarations before reading the design. CARRY4 and SRLC32E stay as named cells. A supported DSP48E2 is changed into GEM\_DSP48E2. This smaller cell carries only the inputs, output, clock, two-bit operation state, and pre-adder selection needed by the project.

The script checks the Yosys version and asks the Slang frontend to read SystemVerilog. It then writes structural Verilog for GEM and JSON for checking. If a raw DSP48E2 remains after the mapping step, the script stops with an error. This prevents an unsupported DSP setup from being accepted by mistake.

The supported DSP operation map is:

| Operation state | OPMODE | Meaning                                           |
|-----------------|--------|---------------------------------------------------|
| 0               | 0x030  | Pass C to the next P value.                       |
| 1               | 0x005  | Write the multiplication result.                  |
| 2               | 0x025  | Add the multiplication result to the old P value. |

### 4.2 Typed macro records

The host code does not depend on the order in which ports appear in a netlist. Every macro input and output has a fixed port type and bit number. For example, DSP A has 27 input bits, DSP B has 18 bits, and DSP P has up to 48 output bits. The code checks the expected width and rejects duplicate or unknown port bits.

Each connection also has a timing type:

- **same cycle:** a combinational macro output feeds a combinational input;
- **clock boundary:** a registered output or next-state input crosses the global edge.

This difference is needed because CARRY4 outputs can affect later work in the same cycle, while the DSP P register changes only at the global clock edge.

### 4.3 Host memory layout

Normal Boolean state stays packed in 32-bit words. Macro state and macro I/O use separate arrays of 64-bit words. Instances are grouped by type and padded to a full warp of 32 entries.

For macro type  $t$ , let  $n_t$  be the number of instances. Its field stride is

$$s_t = 32 \left\lceil \frac{n_t}{32} \right\rceil.$$

For field  $f$  and instance  $i$ , the byte address is

$$A(t, f, i) = B_t + 8(f s_t + i),$$

where  $B_t$  is the start of that macro type. Neighbouring GPU lanes therefore read neighbouring 64-bit words for the same field.

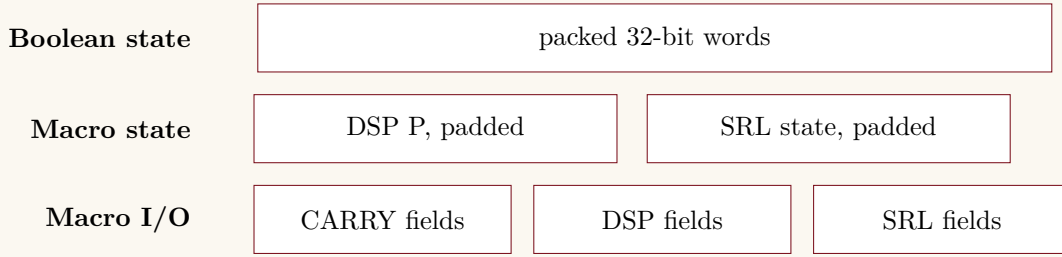


Figure 2: Separate Boolean and word-level memory areas.

#### 4.4 Host formatter and GPU memory hierarchy

The host formatter writes fixed-size macro descriptors containing the macro type, dependency level, bit connections, and memory offsets. It then allocates `macro_state_data` and `macro_io_data` as arrays of `u64`. The CUDA entry point receives these pointers directly. The test runner checks that both device addresses satisfy 64-bit alignment before starting the kernel.

Table 4: Mapping of each primitive to the GPU memory hierarchy.

| Primitive             | Global memory                                                 | Shared memory                             | Registers                                          |
|-----------------------|---------------------------------------------------------------|-------------------------------------------|----------------------------------------------------|
| CARRY4 or fused chain | S, DI, CIN and output words in the 64-bit I/O area            | Current level fields for the active block | Carry addition and O/CO masks                      |
| DSP48E2               | A, D, B, C and controls in I/O; P in state                    | Input fields and visible P value          | Signed 64-bit pre-add, multiply, add and selection |
| SRLC32E               | D, CE and address in I/O; 32-bit shift state in a 64-bit slot | Controls and visible shift state          | Indexed Q read and masked next-state value         |
| Boolean AIG           | Packed 32-bit state and normal GEM stage data                 | Existing Boomerang scan storage           | Per-thread Boolean values and scan values          |

## 5. Deliverable B: CUDA Engine and Boomerang Changes

### 5.1 Macro dependency levels

The host makes a graph from same-cycle macro connections. If  $\text{pred}(v)$  is the set of same-cycle macro nodes before node  $v$ , its level is

$$L(v) = \begin{cases} 0, & \text{pred}(v) = \emptyset, \\ 1 + \max_{u \in \text{pred}(v)} L(u), & \text{otherwise.} \end{cases}$$

Kahn’s topological method is used to build these levels. If nodes remain after the method finishes, the graph contains a same-cycle macro loop and is rejected.

Let  $T(v)$  be the primitive type of macro node  $v$ . The work group for type  $t$  at level  $k$  is

$$M_{t,k} = \{v \mid T(v) = t \text{ and } L(v) = k\}.$$

The host sorts descriptors by  $(T(v), L(v))$ . For one cycle, let  $B(x)$  be one normal Boolean Boomerang pass,  $G_k$  gather the macro inputs for level  $k$ ,  $F_k$  evaluate the three typed groups, and  $W_k$  publish their output bits. The mixed-width recurrence is

$$x_0 = B(x_{in}), \quad x_{k+1} = B(W_k(F_k(G_k(x_k, q_c)))) ,$$

where  $q_c$  is the macro state at cycle  $c$ . Stateful inputs are gathered only after the last same-cycle level, and the single clock commit is

$$q_{c+1} = C(q_c, G_{seq}(x_{last})) .$$

This keeps 1-bit Boolean work in its packed path while a typed group performs word-level work. Grid barriers separate publish and gather steps. Within a typed group, lanes do not select different primitive functions.

Connected CARRY4 cells may be joined into one carry-chain node. A join is used only when CO[3] feeds the next CIN, the later CYINIT is not active, and the total width is no more than 60 bits. This reduces the number of separate carry steps.

### 5.2 Work done in one simulation cycle

The CUDA kernel uses a cooperative launch so all blocks can meet at grid-wide barriers. A cycle follows these main steps:

1. Publish the old DSP P values.
2. Run the normal Boomerang Boolean stages.
3. For each macro level, gather inputs, run one word-level operation per active thread, and write output bits.
4. Run the Boolean stages again so the new macro outputs can move through normal logic.
5. Gather DSP and SRL next-state inputs from one settled view.
6. Update DSP and SRL state once at the global rising edge.

7. Repeat the level walk so the new registered values are visible in the same output cycle.

When a design contains no macros, the code keeps the original Boolean-only Boomerang path. When a circuit contains only combinational CARRY4 blocks, the second state-update pass is skipped because there is no DSP or SRL state to commit. This last change is part of the CARRY4 improvement described later.

### 5.3 Reducing warp divergence

Macro descriptors are sorted by primitive type. The kernel calls a separate typed range for carry, DSP, and SRL work. A warp ballot selects the active instances for the current level. The active lanes therefore use the same macro function in most warps.

DSP operation selection and SRL clock enable use masks instead of different control-flow branches for each lane. The inputs and state are first read from global memory into a shared-memory tile and then used from registers by the macro function. The saved build check confirms the shared tile, 64-bit shared loads and stores, and a warp vote in the compiled kernel.

## 6. Deliverable C: Hardware Macro Models

### 6.1 DSP48E2 subset

Inputs A and D are 27-bit signed values. B is an 18-bit signed value. C and P are 48-bit signed values. If  $u$  selects the pre-adder, then

$$AD = [A + uD]_{27}, \quad M = \text{signed}(AD) \times \text{signed}(B).$$

At the rising edge, the next P value is

$$P_{next} = \begin{cases} C, & state = 0, \\ M, & state = 1, \\ P + M, & state = 2. \end{cases}$$

The result is cut to 48 bits. The CUDA function uses `gem_i64`, a native signed 64-bit type, which is large enough for the required 45-bit product.

### 6.2 CARRY4 and fused carry chains

For each bit  $i$ , the model follows the problem statement:

$$\begin{aligned} C_0 &= CYINIT \vee CIN, \\ C_{i+1} &= (S_i \wedge C_i) \vee (\neg S_i \wedge DI_i), \\ O_i &= S_i \oplus C_i, \quad CO_i = C_{i+1}. \end{aligned}$$

The fused chain uses one 64-bit addition to produce the result for up to fifteen CARRY4 blocks. The maximum width is kept at 60 bits so the final carry also fits inside the 64-bit calculation.

### 6.3 SRLC32E

Let  $R$  be the 32-bit state. On a rising edge:

$$R_{next} = \begin{cases} [(R \ll 1) \vee D]_{32}, & CE = 1, \\ R, & CE = 0. \end{cases}$$

The two combinational outputs are

$$Q = R[A], \quad Q31 = R[31].$$

The state array starts at zero. The address can change during a cycle, so the Q read is included in the same-cycle macro schedule.

### 6.4 C++ and CUDA implementation files

The macro work is not only a report-level model. The reusable C++/CUDA functions are stored in `csrc/gem_macros.cuh`. The production simulator calls them from `csrc/kernel_v1_impl.cuh`. The direct CUDA check in `verif/csrc/test_gem_macros.cu` sends test vectors to the GPU and compares every result with the reference values. DSP P and SRL state are checked after each simulated

rising edge. This is cycle-accurate for the supported primitive subset; it is not a propagation-delay model.

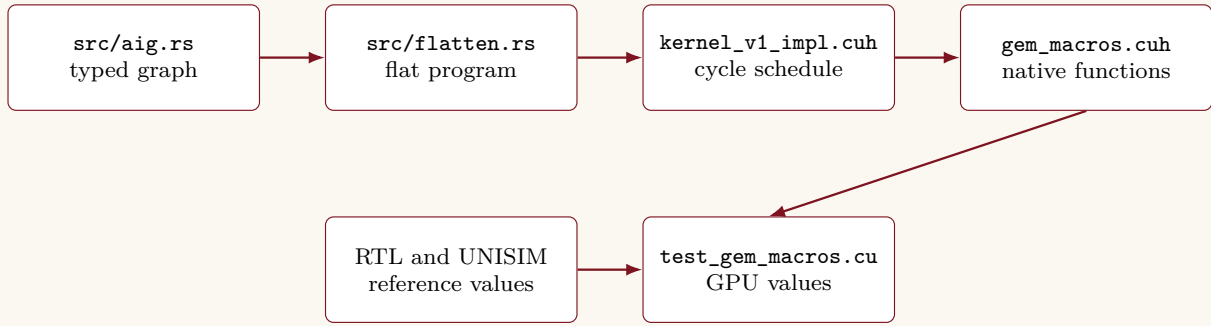


Figure 3: Production code path and direct CUDA verification path.

Cycle accuracy is checked at three points: combinational outputs after the Boolean and macro schedule settles, DSP P at the global rising edge, and SRL state at the same edge. The production mixed-chain tests repeat these checks with one block, four blocks, and a split Boomerang stage.

## 7. Verification Work

### 7.1 Levels of checking

The project uses more than one kind of check:

- Python models check the required bit equations.
- Verilog testbenches check clock and port behaviour.
- Xilinx UNISIM is used when the Vivado model files are available.
- A CUDA test runs the same vectors on the GPU macro functions.
- Rust tests check parsing, graph creation, carry-chain linking, and memory offsets.
- Full-path tests compare RTL results with output from the real GEM CUDA flow.

### 7.2 Saved test results

Table 5: Saved verification results.

| Check                       | Result | Saved coverage                                                                   |
|-----------------------------|--------|----------------------------------------------------------------------------------|
| CARRY4 slice                | PASS   | 3,024 vectors, including the complete 1,024 input cases.                         |
| 60-bit fused carry chain    | PASS   | 2,009 vectors.                                                                   |
| DSP48E2                     | PASS   | 2,336 sequential vectors.                                                        |
| SRLC32E                     | PASS   | 2,100 sequential vectors.                                                        |
| Xilinx UNISIM comparison    | PASS   | All three required primitive types.                                              |
| Exact mixed chain           | PASS   | DSP to CARRY4 to SRLC32E to DSP; 2,709 values checked on one and four blocks.    |
| Random mixed graphs         | PASS   | Four generated topologies and 192 stimulus cycles; 1,029 values checked per run. |
| Compiled CUDA feature check | PASS   | Shared tile, 64-bit shared access, and warp vote found.                          |

The main verification command is:

```
python3 verif/full_integration_test.py
```

A fresh run of this command was completed after changing the macro aliases to standard fixed-width integer types. All nine phases passed, including the full workspace tests, exact mixed chain, four randomized mixed topologies, and the production Nsight check. The submission archive should still be made from a clean commit so its commit ID matches the saved benchmark metadata.

## 8. Deliverable D: Benchmarks and Performance Analysis

### 8.1 Test method

The main unit is simulated cycles per second:

$$CPS = \frac{\text{number of simulated cycles}}{\text{synchronised CUDA launch time in seconds}}.$$

The saved main test uses twelve warm-up launches and seven measured launches. The median of the seven samples is used in this report. Synthesis, graph building, memory allocation, file transfer, and output writing are outside the timed part. This makes the number a kernel-throughput result, not a complete command run time.

The measured machine used an NVIDIA GeForce RTX 4050 Laptop GPU with compute capability 8.9 and 6,141 MiB of memory. The saved environment also records CUDA 12.9, Nsight Compute 2025.2.1, Yosys 0.68, and Rust 1.98.0.

### 8.2 Modified GEM throughput

Table 6: Median throughput from the saved main benchmark.

| Workload            | Cycles | Median CPS | Main content                       |
|---------------------|--------|------------|------------------------------------|
| Boolean heavy       | 2,000  | 73,332     | Boolean AIG only                   |
| DSP heavy           | 2,000  | 21,226     | 32 DSP48E2 blocks                  |
| CARRY heavy         | 1,000  | 3,163      | 128 CARRY4 blocks                  |
| SRL heavy           | 2,000  | 43,784     | 128 SRLC32E blocks                 |
| Small mixed         | 1,000  | 7,593      | 4 DSP, 8 CARRY4, 8 SRL             |
| Mixed heterogeneous | 1,000  | 3,523      | All three macro types              |
| Deep dependency     | 200    | 896        | 64 CARRY4 blocks in deeper logic   |
| Large scale         | 200    | 1,036      | 32 DSP, 128 CARRY4, 128 SRL        |
| Occupancy stress    | 1,000  | 16,762     | Wide output design, 16 CUDA blocks |

These rows compare different circuits, so they must not be treated as speedups against each other. They show which types of circuits cost more in the current kernel. The carry-heavy and deep-dependency cases are the slowest groups.

### 8.3 Fair upstream Boolean comparison

The saved paired comparison uses the same Boolean netlist, cycle count, and block count. Each program uses the partition file made by its own compatible mapper. The result is:

| Version                 | Median CPS |
|-------------------------|------------|
| Unmodified upstream GEM | 61,709     |
| Modified Big-GEM        | 55,125     |
| Modified / upstream     | 0.893×     |
| Change                  | −10.67%    |

Table 7: Identical Boolean workload comparison.

The standalone Boolean row in the previous table has different run settings and is therefore not used for this upstream ratio. The paired result is the honest comparison. It shows that the current changes add some overhead even when the circuit has no macros.

#### 8.4 CARRY4 bottleneck work

The CARRY4 test compares the same combinational RTL in two forms: a shredded upstream netlist and a macro-preserved Big-GEM netlist. This test changes both the representation and the simulator, so it is not an implementation-only speedup.

| Version                        | Median CPS      |
|--------------------------------|-----------------|
| Upstream GEM, shredded CARRY4  | 80,033          |
| Big-GEM, preserved CARRY4      | 17,734          |
| Current ratio                  | 0.2216×         |
| Old ratio                      | 0.117×          |
| Improvement over the old ratio | 1.894× or 89.4% |

Table 8: Saved CARRY4 comparison.

The current result is stored in `benchmark/results/carry4_optimization.json`. The old ratio is kept in the CARRY4 benchmark script. The change skips unused DSP work when no DSP blocks exist. It also uses only one level walk when a circuit has combinational CARRY4 blocks but no DSP or SRL state. This nearly doubles the old ratio. However, Big-GEM is still about 77.8% slower than the upstream shredded form in this test.

#### 8.5 Nsight Compute results

Table 9: Saved Nsight Compute summaries.

| Profile          | Warp divergence | DRAM peak use | DRAM rate  | Kernel time |
|------------------|-----------------|---------------|------------|-------------|
| Upstream Boolean | 1.94%           | 0.01%         | 14.10 MB/s | 41.817 ms   |
| Modified Boolean | 1.89%           | 0.01%         | 13.01 MB/s | 45.635 ms   |
| Modified mixed   | 1.19%           | below 0.01%   | 1.98 MB/s  | 472.534 ms  |

The low divergence values show that most branch targets are uniform. The low DRAM percentage shows that memory bandwidth is not close to the GPU limit in these small profiles. The mixed

kernel takes much longer even though it does not use much DRAM bandwidth. This points to repeated level walks, barriers, small active macro groups, and instruction cost as more important limits than raw DRAM bandwidth.

The modified kernel uses 121 registers per thread and 16,640 bytes of shared memory per block in the saved profiles. Upstream uses 90 registers per thread and 4,352 bytes of shared memory. Both saved Boolean profiles report about 16.7% achieved occupancy. These results suggest that reducing the number of schedule passes and the per-thread state should be tested before adding more memory movement.

## 8.6 Numerical analysis

The seven repetitions are stable. The coefficient of variation is 0.17% for the Boolean workload, 0.04% for DSP, 0.01% for CARRY, 0.38% for SRL, and below 0.08% for every other modified workload. Therefore, the large gaps in the table are much greater than normal run-to-run noise.

Using the standalone Boolean result only as a throughput scale, DSP reaches 28.9% of its CPS, SRL reaches 59.7%, CARRY reaches 4.3%, and the mixed case reaches 4.8%. These are not speedups because the circuits have different cell counts. They show that dependency depth and repeated scheduling cost much more than the native arithmetic itself.

The paired Boolean test gives a direct comparison. Big-GEM loses 6,584 CPS, or 10.67%, on the same input. The profiler shows a similar cost: modified Boolean takes 3.818 ms longer, an increase of 9.1%. Registers rise from 90 to 121 per thread, and shared memory rises from 4,352 to 16,640 bytes per block. Divergent branch targets improve slightly from 1.94% to 1.89%, so branch divergence does not explain the slowdown.

In the CARRY4 representation experiment, preservation reduces the netlist from 850 to 709 cells, a 16.6% reduction. AIG cells fall from 785 to 628, a 20.0% reduction, while 15 CARRY4 cells remain. Even with fewer cells, the preserved path reaches only 22.2% of upstream CPS. This is strong evidence that the main cost is the current level and barrier schedule, not the number of Boolean gates.

The mixed profile takes 472.534 ms, which is 10.4 times the modified Boolean profile, while its DRAM rate is only 1.98 MB/s. This supports the same finding: future work should reduce full Boolean passes, grid barriers, and partly filled macro groups before attempting more memory-bandwidth changes.

---

## 9. Deliverable E: Documentation and Repository Work

---

### 9.1 Documentation added

The repository keeps NVIDIA’s original README and adds a separate `ZENITH_SETUP.md` for the Zenith build and test setup. The benchmark folder has its own short README with the three main commands. Verification results are listed under `verif/results/`.

This report adds the required scheduling equations, the CPU-to-GPU data path, the memory layout, macro equations, test results, and performance analysis.

---

## 10. Build and Reproduction Steps

---

### 10.1 Required tools

The project needs Linux, an NVIDIA GPU, CUDA and `nvcc`, Rust and Cargo, Python 3, Yosys 0.68 with Slang, and Nsight Compute. Icarus Verilog is used by the verification scripts. Xilinx UNISIM source files are optional but needed for the vendor-model comparison.

### 10.2 Build

```
cargo build --release --features cuda
```

The CUDA target can be set when automatic detection is not suitable:

```
GEM_CUDA_ARCH=sm_89 cargo build --release --features cuda
```

### 10.3 Run verification

```
python3 veril/full_integration_test.py
```

### 10.4 Run benchmarks

```
python3 benchmark/scripts/run_benchmarks.py --repetitions 7
python3 benchmark/scripts/benchmark_carry4.py --repetitions 7 --blocks 4
python3 benchmark/scripts/profile_ncu.py --workload upstream_boolean
python3 benchmark/scripts/profile_ncu.py --workload boolean_heavy
python3 benchmark/scripts/profile_ncu.py --workload mixed_heterogeneous
```

Nsight Compute needs permission to read NVIDIA performance counters. The script reports a clear error if that permission is missing.

---

## 11. Performance limits

---

- The current Boolean comparison is 10.67% slower than upstream.
- The CARRY4 path improved, but it is still much slower than upstream.
- A heterogeneous cycle may repeat the Boolean level walk many times.
- Small macro groups leave many CUDA lanes with no useful macro work.
- The modified kernel uses more registers and shared memory than upstream.

The first performance task should be to reduce repeated full Boolean passes. A possible next step is to run only the Boolean parts affected by one macro level. The second task is to group more independent macro instances into full warps. Both changes must be checked with the existing mixed-chain tests before their speed is measured.

---

## 12. Conclusion

---

Big-GEM now keeps DSP48E2, CARRY4, and SRLC32E cells through synthesis and runs them inside the production CUDA simulation path. The host has typed ports, dependency levels, carry-chain joining, and separate aligned macro memory. The GPU has native word-level functions, shared-memory staging, typed dispatch, and one global state-update point.

The saved checks support the functional work. The performance results are more mixed. Warp divergence is low, but the repeated scheduling work makes macro-heavy circuits slow. The CARRY4 change improves the earlier result, yet it does not beat upstream. The project is therefore a working heterogeneous simulator prototype, with performance optimisation still needed.

---

## 13. References

---

1. Takneek 2026 Zenith problem statement, “The Big-GEM Theory.”
2. Z. Guo, Y. Zhang, R. Wang, Y. Lin, and H. Ren, “GEM: GPU-Accelerated Emulator-Inspired RTL Simulation,” Design Automation Conference, 2025.
3. NVIDIA Research, GEM source repository: <https://github.com/NVlabs/GEM>.
4. Yosys 0.68 and Slang command help used by the installed synthesis flow.

## A. Deliverable-to-File Map

| Part | Main file                                          | What it shows                                                        |
|------|----------------------------------------------------|----------------------------------------------------------------------|
| A    | <code>scripts/synthesize_macros.py</code>          | Yosys version check, macro preservation, DSP map, and output checks. |
| A    | <code>aigpdk/dsp48e2_ps_map.v</code>               | Supported DSP48E2 parameter and control mapping.                     |
| A    | <code>src/aig.rs</code>                            | Named macro ports and graph construction.                            |
| A    | <code>src/macro_layout.rs</code>                   | Typed dependencies and 64-bit memory layout.                         |
| B    | <code>src/flatten.rs</code>                        | Flat macro descriptors and level data for CUDA.                      |
| B    | <code>csrc/kernel_v1_impl.cuh</code>               | Combined Boomerang and macro schedule.                               |
| C    | <code>csrc/gem_macros.cuh</code>                   | Native CUDA macro equations.                                         |
| C    | <code>verif/golden/</code>                         | Independent Python models.                                           |
| C    | <code>verif/tb/</code>                             | Verilog and UNISIM testbenches.                                      |
| D    | <code>benchmark/scripts/run_benchmarks.py</code>   | CPS collection and upstream Boolean comparison.                      |
| D    | <code>benchmark/scripts/benchmark_carry4.py</code> | CARRY4 comparison and result writing.                                |
| D    | <code>benchmark/scripts/profile_ncu.py</code>      | Nsight Compute command and summary generation.                       |
| E    | <code>ZENITH_SETUP.md</code>                       | Setup and main commands.                                             |
| E    | <code>report/Report_Latex.tex</code>               | Source of this report.                                               |

## B. Evidence Folders

The performance files used in this report are stored in `benchmark/results/` and `benchmark/profiles/`. The saved test logs are stored in `verif/results/`.